

Sto creando un progetto typescript per un giochino arcade html5. Ci saranno alcuni pezzi di codice che disegneranno su un canvas nell html, mentre altri pezzi di codice che si occupano della logica pura del gioco. Come organizzare il progetto? Sii più dettagliato possibile

Per un gioco arcade HTML5 in TypeScript, una buona organizzazione iniziale ti evita molti problemi quando il progetto cresce. Il principio fondamentale è **separare la logica del gioco dalla presentazione (rendering)**.

Una struttura che funziona molto bene è quella a livelli:

```
src/
|
├─ core/                # motore di gioco generico
|   ├── Game.ts
|   ├── GameLoop.ts
|   ├── Scene.ts
|   ├── Input.ts
|   └─ Time.ts
|
├─ game/                # regole specifiche del tuo gioco
|   ├── Player.ts
|   ├── Enemy.ts
|   ├── Bullet.ts
|   ├── Collision.ts
|   ├── GameState.ts
|   └─ systems/
|
├─ render/              # tutto ciò che disegna
|   ├── Renderer.ts
|   ├── CanvasRenderer.ts
|   ├── SpriteRenderer.ts
|   └─ Camera.ts
|
├─ assets/
|   ├── images/
|   ├── sounds/
|   └─ fonts/
|
├─ ui/
|   ├── Hud.ts
|   ├── ScoreDisplay.ts
|   └─ Menu.ts
|
├─ utils/
|   ├── Vector2.ts
|   ├── Random.ts
|   └─ MathUtils.ts
|
├─ types/
|   └─ GameTypes.ts
|
└─ main.ts
```

1. Separare modello e vista

L'errore più comune è mescolare:

TypeScript

```
player.x += 10;  
  
ctx.fillRect(player.x, player.y, 32, 32);
```

La logica e il rendering finiscono insieme.

Meglio:

TypeScript

```
// LOGICA  
player.moveRight();
```

e altrove:

TypeScript

```
renderer.drawPlayer(player);
```

Così il giocatore non sa nulla del canvas.

2. Il dominio del gioco

Le classi che rappresentano gli oggetti del gioco dovrebbero contenere solo stato e comportamento.

Esempio:

TypeScript

```
export class Player {  
    position: Vector2;  
    velocity: Vector2;  
  
    constructor() {  
        this.position = new Vector2(100, 100);  
        this.velocity = new Vector2();  
    }  
  
    update(deltaTime: number) {  
        this.position.add(  
            this.velocity.scaled(deltaTime)  
        );  
    }  
}
```

Nessun riferimento a:

TypeScript

CanvasRenderingContext2D

HTMLCanvasElement

Image

Queste appartengono al rendering.

3. Rendering layer

Un renderer riceve dati e li disegna.

TypeScript

```

export class PlayerRenderer {

    draw(
        ctx: CanvasRenderingContext2D,
        player: Player
    ) {
        ctx.fillStyle = "red";

        ctx.fillRect(
            player.position.x,
            player.position.y,
            32,
            32
        );
    }
}

```

Oppure un renderer unico:

TypeScript

```

renderer.render(world);

```

che internamente disegna tutto.

4. Game Loop

Quasi tutti gli arcade hanno una struttura simile.

TypeScript

```

class GameLoop {

    start() {

        const tick = (time: number) => {

            this.update();

            this.render();

            requestAnimationFrame(tick);
        };

        requestAnimationFrame(tick);
    }
}

```

Separazione netta:

TypeScript

```
update()
```

per la simulazione

e

TypeScript

```
render()
```

per il disegno.

5. Oggetti di gioco

Puoi definire una classe base.

TypeScript

```
export abstract class Entity {  
  
    position: Vector2;  
  
    abstract update(dt: number): void;  
}
```

Poi:

TypeScript

```
export class Player extends Entity  
export class Enemy extends Entity  
export class Bullet extends Entity
```

6. World

Invece di avere liste sparse ovunque:

TypeScript

```
players
enemies
bullets
coins
```

crea un contenitore.

TypeScript

```
export class World {

    players: Player[] = [];

    enemies: Enemy[] = [];

    bullets: Bullet[] = [];
}
```

Il loop lavora sul world.

TypeScript

```
world.update(dt);
renderer.render(world);
```

7. Input System

L'input non dovrebbe modificare direttamente il player.

Evita:

TypeScript

```
window.addEventListener("keydown", () => {
    player.x += 10;
});
```

Meglio:

TypeScript

```
input.isKeyPressed("ArrowRight")
```

e poi:

TypeScript

```
player.handleInput(input);
```

Esempio:

TypeScript

```
export class Input {  
  
    private keys = new Set<string>();  
  
    isPressed(key: string): boolean {  
        return this.keys.has(key);  
    }  
}
```

8. Sistema di collisione

Mantienilo separato.

TypeScript

```
export class CollisionSystem {  
  
    check(  
        bullets: Bullet[],  
        enemies: Enemy[]  
    ) {  
  
        // collisioni  
    }  
}
```

Nel loop:

TypeScript

```
collisionSystem.update(world);
```

9. Asset Manager

Quando arriveranno immagini e suoni ti servirà.

TypeScript

```
export class AssetManager {  
  
    private images = new Map<string, HTMLImageElement>();  
  
    async loadImage(  
        id: string,  
        path: string  
    ) {  
        ...  
    }  
  
    getImage(id: string) {  
        ...  
    }  
}
```

Così eviti:

TypeScript

```
new Image()
```

sparsi in tutto il progetto.

10. Scene

Molti giochi hanno stati diversi:

- menu
- partita
- game over
- pausa

Una struttura a scene è molto utile.

TypeScript

```
abstract class Scene {  
  
    abstract update(dt: number): void;  
  
    abstract render(): void;  
}
```

Poi:

TypeScript

MenuScene

GameScene

PauseScene

GameOverScene

11. Eventi

Per evitare accoppiamento forte:

TypeScript

```
enemy.onDeath(() => {  
    score += 100;  
});
```

oppure un event bus:

TypeScript

```
eventBus.emit("enemyKilled");
```

e

TypeScript

```
eventBus.on("enemyKilled", ...);
```

12. ECS (solo se il progetto cresce)

Per un piccolo arcade NON partire con ECS.

Inizia con:

TypeScript

Player

Enemy

Bullet

come classi normali.

Passa a ECS solo se arrivi a centinaia/migliaia di entità o a meccaniche molto complesse.

13. Esempio di flusso completo

```
main.ts
|
v
Game
|
+--> Input
|
+--> World
|
+--> CollisionSystem
|
+--> Renderer
```

Ad ogni frame:

1. Leggi input
2. Aggiorna player
3. Aggiorna nemici
4. Aggiorna proiettili
5. Calcola collisioni
6. Aggiorna punteggio
7. Disegna tutto

14. Struttura consigliata per un piccolo arcade

Per un progetto che deve rimanere semplice:

```
src/
|
├─ core/
|   ├── Game.ts
|   ├── GameLoop.ts
|   └─ Input.ts
|
├─ game/
|   ├── entities/
|   |   ├── Player.ts
|   |   ├── Enemy.ts
|   |   └─ Bullet.ts
|   |
|   ├── systems/
|   |   ├── CollisionSystem.ts
|   |   └─ SpawnSystem.ts
|   |
|   └─ World.ts
|
├─ render/
|   ├── Renderer.ts
|   └─ SpriteRenderer.ts
|
├─ assets/
|
├─ utils/
|   └─ Vector2.ts
|
└─ main.ts
```

È abbastanza pulita da supportare mesi di sviluppo, ma non così complessa da diventare un framework prima ancora di avere il gioco. La regola principale è: **le classi di gioco non devono conoscere il canvas, e il renderer non deve contenere regole di gameplay**. Questa separazione rende testabile la logica e molto più semplice aggiungere nuove funzionalità.

